



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO
NACIONAL DE MÉXICO®



INSTITUTO TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE TIJUANA

MATERIA

Patrones de diseño

NOMBRE DEL TRABAJO

Examen

UNIDAD 2

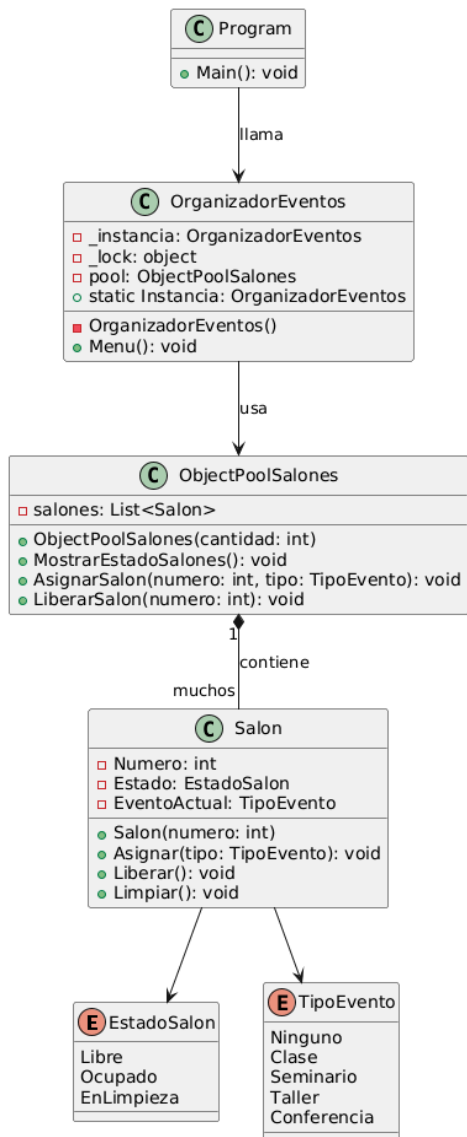
ALUMNO

Sandoval Ortiz Andy - 21210547

MAESTRO

Maribel Guerrero Luis

1. Diagrama UML



2. Código

```
7 referencias
enum EstadoSalon
{
    Libre,
    Ocupado,
    EnLimpieza
}

12 referencias
enum TipoEvento
{
    Ninguno,
    Clase,
    Seminario,
    Taller,
    Conferencia
}

4 referencias
class Salon
{
    7 referencias
    public int Numero { get; private set; }
    9 referencias
    public EstadoSalon Estado { get; set; }
    4 referencias
    public TipoEvento EventoActual { get; set; }

    1 referencia
    public Salon(int numero)
    {
        Numero = numero;
        Estado = EstadoSalon.Libre;
        EventoActual = TipoEvento.Ninguno;
    }

    1 referencia
    public void Asignar(TipoEvento tipo)
    {
        Estado = EstadoSalon.Ocupado;
        EventoActual = tipo;
        Console.WriteLine($"Salón {Numero} asignado para un evento: {tipo}.");
    }

    1 referencia
    public void Liberar()
    {
        Estado = EstadoSalon.EnLimpieza;
        Console.WriteLine($"Salón {Numero} ha sido liberado y está en limpieza.");
    }

    1 referencia
    public void Limpiar()
    {
        Estado = EstadoSalon.Libre;
        EventoActual = TipoEvento.Ninguno;
        Console.WriteLine($"Salón {Numero} ha sido limpiado y está disponible.");
    }
}
```

3 referencias

```
class ObjectPoolSalones
{
    private readonly List<Salon> salones;

    1 referencia
    public ObjectPoolSalones(int cantidad)
    {
        salones = new List<Salon>();
        for (int i = 1; i <= cantidad; i++)
        {
            salones.Add(new Salon(i));
        }
    }

    1 referencia
    public void MostrarEstadoSalones()
    {
        Console.WriteLine("\nEstado actual de los salones:");
        foreach (var s in salones)
        {
            Console.WriteLine($" - Salón {s.Numero}: {s.Estado}, Evento: {s.EventoActual}");
        }
        Console.WriteLine();
    }

    1 referencia
    public void AsignarSalon(int numero, TipoEvento tipo)
    {
        var salon = salones.FirstOrDefault(s => s.Numero == numero);
        if (salon == null)
        {
            Console.WriteLine("El salón no existe.");
            return;
        }

        if (salon.Estado == EstadoSalon.Libre)
        {
            salon.Asignar(tipo);
        }
        else
        {
            Console.WriteLine($"El salón {numero} no está disponible. Estado actual: {salon.Estado}");
        }
    }
}
```

1 referencia

```
public void LiberarSalon(int numero)
{
    var salon = salones.FirstOrDefault(s => s.Numero == numero);
    if (salon == null)
    {
        Console.WriteLine("El salón no existe.");
        return;
    }

    if (salon.Estado == EstadoSalon.Ocupado)
    {
        salon.Liberar();

        Thread limpiezaThread = new Thread(() =>
        {
            Thread.Sleep(3000);
            salon.Limpiar();
        });

        limpiezaThread.Start();
        limpiezaThread.Join();
    }
    else
    {
        Console.WriteLine($"El salón {numero} no puede liberarse. Estado actual: {salon.Estado}");
    }
}
```

5 referencias

class OrganizadorEventos

```
{
    private static OrganizadorEventos _instancia;
    private static readonly object _lock = new object();
    private ObjectPoolSalones pool;
```

1 referencia

private OrganizadorEventos()

```
{
    pool = new ObjectPoolSalones(5);
}
```

1 referencia

public static OrganizadorEventos Instancia

```
{
    get
    {
        if (_instancia == null)
        {
            lock (_lock)
            {
                if (_instancia == null)
                {
                    _instancia = new OrganizadorEventos();
                }
            }
        }
        return _instancia;
    }
}
```

```

public void Menu()
{
    while (true)
    {
        Console.WriteLine("\n--- MENÚ ORGANIZADOR DE EVENTOS ACADÉMICOS ---");
        Console.WriteLine("1. Ver estado de salones");
        Console.WriteLine("2. Asignar salón para evento");
        Console.WriteLine("3. Liberar salón (limpieza automática)");
        Console.WriteLine("4. Salir");
        Console.Write("Seleccione una opción: ");
        var opcion = Console.ReadLine();

        switch (opcion)
        {
            case "1":
                pool.MostrarEstadoSalones();
                break;

            case "2":
                Console.Write("Ingrese número del salón a asignar: ");
                if (int.TryParse(Console.ReadLine(), out int numAsignar))
                {
                    Console.WriteLine("Seleccione tipo de evento:");
                    foreach (var tipo in Enum.GetValues(typeof(TipoEvento)))
                    {
                        if ((TipoEvento)tipo != TipoEvento.Ninguno)
                            Console.WriteLine($"{(int)tipo} = {tipo}");
                    }

                    if (int.TryParse(Console.ReadLine(), out int tipoEvento) &&
                        Enum.IsDefined(typeof(TipoEvento), tipoEvento) &&
                        (TipoEvento)tipoEvento != TipoEvento.Ninguno)
                    {
                        pool.AsignarSalon(numAsignar, (TipoEvento)tipoEvento);
                    }
                    else
                    {
                        Console.WriteLine("Tipo de evento inválido.");
                    }
                }
                else
                {
                    Console.WriteLine("Número inválido.");
                }
                break;

            case "3":
                Console.Write("Ingrese número del salón a liberar: ");
                if (int.TryParse(Console.ReadLine(), out int numLiberar))
                    pool.LiberarSalon(numLiberar);
                else
                    Console.WriteLine("Número inválido.");
                break;

            case "4":
                Console.WriteLine("Saliendo del sistema...");
                return;

            default:
                Console.WriteLine("Opción inválida. Intente de nuevo.");
                break;
        }
    }
}

```

3. Código Funcional

```
E:\Codigos\Patrones de Diseñ  X  +  v

--- MENÚ ORGANIZADOR DE EVENTOS ACADÉMICOS ---
1. Ver estado de salones
2. Asignar salón para evento
3. Liberar salón (limpieza automática)
4. Salir
Seleccione una opción: 2
Ingrese número del salón a asignar: 3
Seleccione tipo de evento:
- 1 = Clase
- 2 = Seminario
- 3 = Taller
- 4 = Conferencia
3
Salón 3 asignado para un evento: Taller.

--- MENÚ ORGANIZADOR DE EVENTOS ACADÉMICOS ---
1. Ver estado de salones
2. Asignar salón para evento
3. Liberar salón (limpieza automática)
4. Salir
Seleccione una opción: 1

Estado actual de los salones:
- Salón 1: Libre, Evento: Ninguno
- Salón 2: Libre, Evento: Ninguno
- Salón 3: Ocupado, Evento: Taller
- Salón 4: Libre, Evento: Ninguno
- Salón 5: Libre, Evento: Ninguno

--- MENÚ ORGANIZADOR DE EVENTOS ACADÉMICOS ---
1. Ver estado de salones
2. Asignar salón para evento
3. Liberar salón (limpieza automática)
4. Salir
Seleccione una opción: 3
Ingrese número del salón a liberar: 3
Salón 3 ha sido liberado y está en limpieza.
Salón 3 ha sido limpiado y está disponible.

--- MENÚ ORGANIZADOR DE EVENTOS ACADÉMICOS ---
1. Ver estado de salones
2. Asignar salón para evento
3. Liberar salón (limpieza automática)
4. Salir
Seleccione una opción: |
```

Conclusión

El código muestra cómo se pueden combinar de manera efectiva los patrones **Singleton** y **Object Pool**, siguiendo buenas prácticas de programación orientada a objetos. El **Singleton** asegura que solo haya un punto de control central llamado `OrganizadorEventos`, y el `Object Pool` ayuda a aprovechar mejor los recursos al reutilizar las instancias de salones en lugar de crearlas cada vez.

Al combinar estos patrones, el sistema se vuelve más eficiente, modular y fácil de mantener. Además, se fomenta la responsabilidad única, el encapsulamiento y el uso claro de enumeraciones, lo que mejora la legibilidad. En conjunto, esta solución es escalable y útil para gestionar recursos compartidos en aplicaciones reales.